

## 04\_CONCORRENZA PARTE 2

Abbiamo visto l'algoritmo Dijkstra:

```

/* global storage */
boolean interested[N] = {false, ..., false}
boolean passed[N]      = {false, ..., false}
int k = <any>           // k ∈ {0, 1, ..., N-1}

/* local info */
int i = <entity ID>     // i ∈ {0, 1, ..., N-1}

while (true) {
    /*NCS*/
    1. interested[i] = true  io sono interessato
    2. while (k != i) {      finché è il turno di qualcun'altro
    3.     passed[i] = false  non sono passato
    4.     if (!interested[k]) then k = i
                                   se anche il processo k è
                                   interessato continuo ad
                                   aspettare, se il processo k
                                   non è interessato allora dico
                                   k=i
                                   Mi prendo quindi il turno
    5.     passed[i] = true   sono passato
    6.     for j in 1 ... N except i do
    7.         if (passed[j]) then goto 2
                                   se c'è anche un solo altro processo
                                   che è passato allora ricomincio da
                                   capo
    8.     <critical section>  sezione critica
    9.     passed[i] = false; interested[i] = false
}

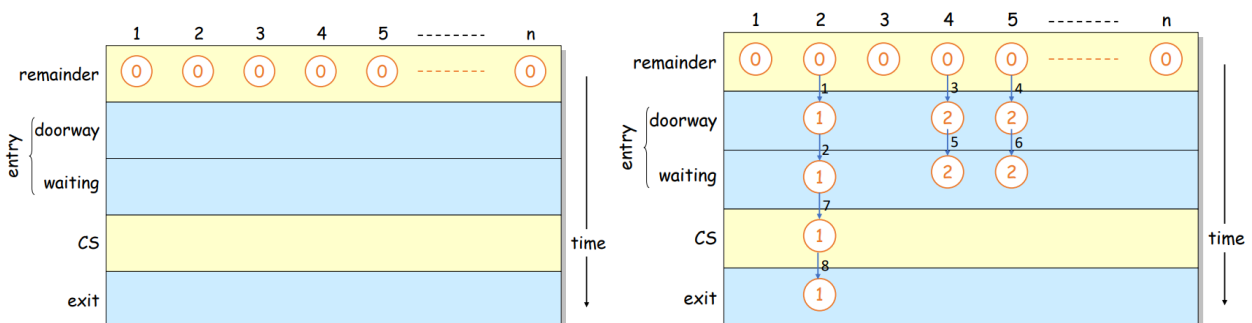
```

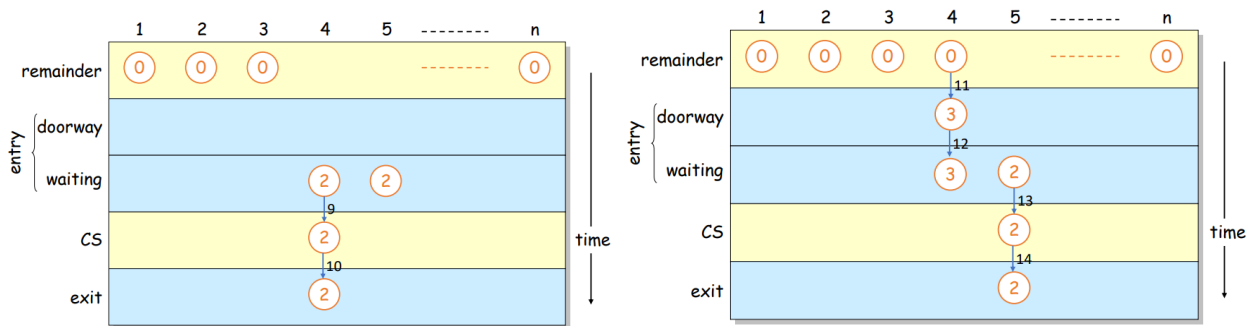
L'algoritmo Dijkstra garantisce la mutua esclusione ed evita deadlock.

La starvation è però ancora possibile ed ha bisogno di read/write atomiche e di memory sharing per la variabile k.

### Bakery Algorithm:

Concettualmente possiamo pensare a un panettiere con un banco affollato. Le persone prendono un biglietto, ma se nessuno sta aspettando i biglietti non contano. Invece quando molte persone stanno aspettando l'ordine dei biglietti determina l'ordine in cui i clienti possono essere serviti.





Vediamo una prima implementazione:

```

while (1){
    /*NCS*/
    number[i] = 1 + max {number[j] | (1 ≤ j ≤ N) except i}
    for j in 1 .. N except i {
        while (number[j] != 0 && number[j] < number[i]);
    }
    /*CS*/ sezione critica
    number[i] = 0; il processo i non ha più il biglietto
}

```

il numero assegnato al processo i è il 1 + il numero massimo assegnato ad un altro processo j ≠ i //Doorway

"prende il biglietto"

per ogni altro processo j ≠ i, finché j è in attesa ed ha un numero più piccolo di i, il processo i aspetta

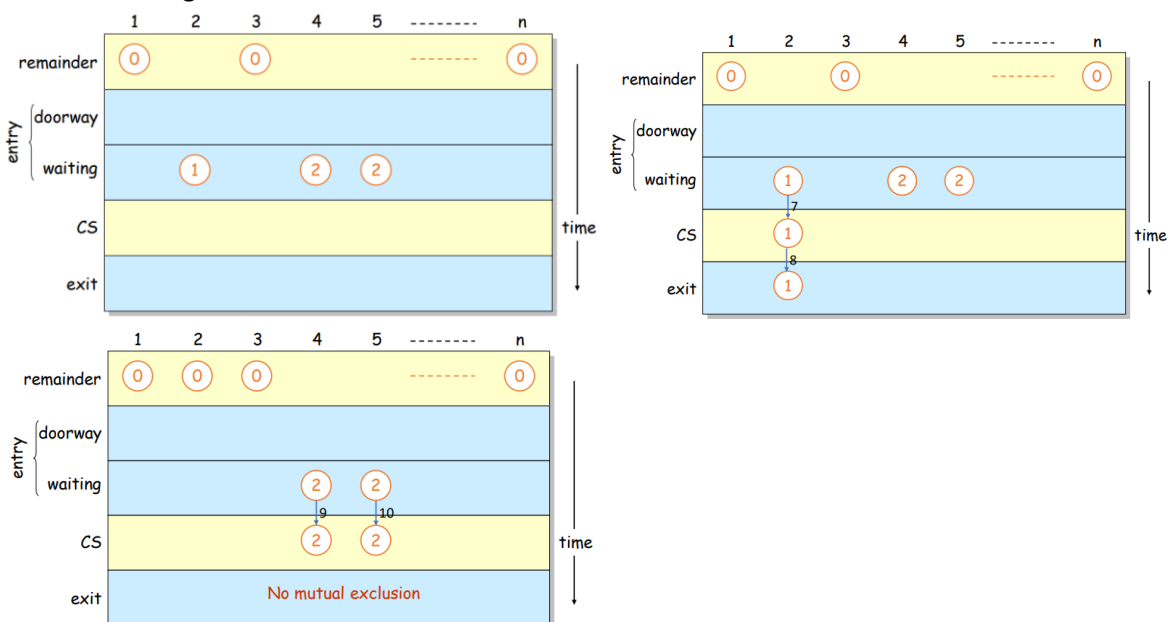
//Bakery

array dei numeri (biglietti) assegnati ad ogni processo:

|        | 1 | 2 | 3 | 4 | ----- | n |
|--------|---|---|---|---|-------|---|
| number | 0 | 0 | 0 | 0 | 0     | 0 |

integer

Problema: non garantisce la mutua esclusione



Vediamo una seconda implementazione:



Per calcolare il massimo (compare nella riga `number[i] = 1 + max {number[j] | (1 ≤ j ≤ N)}`):

```
local1 = 0;
for local2 in 1 .. N {
    local3 = number[local2];
    if (local1 < local3)
        local1 = local3;
}
number[i] = 1 + local1
```

|        |   |   |   |   |       |   |         |
|--------|---|---|---|---|-------|---|---------|
|        | 1 | 2 | 3 | 4 | ----- | n |         |
| number | 0 | 0 | 0 | 0 | 0     | 0 | integer |

Bakery algorithm con numbers limitato:

```
while (1){
    /*NCS*/
    while(number[i] == 0){
        choosing[i] = true;
        number[i] = (1 + max {number[j] | (1 ≤ j ≤ N) except i}) % MAXIMUM;
        choosing[i] = false;
    }
    for j in 1 .. N except i {
        while (choosing[j] == true);
        while (number[j] != 0 && (number[j],j) < (number[i],i));
    }
    /*CS*/
    number[i] = 0;
}
```

Caratteristiche del Bakery algorithm:

- Processes communicate by writing/reading shared variables (as Dijkstra)
- Read/write are not atomic operations, reader can read while writer is writing and none receives any notification
- Any shared variable is owned by a process that can write it, others can read it
- No process can perform two concurrent writings
- Execution times are not correlated

Bakery algorithm in applicazione client/server:

```

while (1){ //client thread
/*NCS*/
choosing = true; //doorway
for j in 1 .. N except i {
    send(Pj,num);
    receive(Pj,v);      chiedo a tutti gli altri processi di
                        mandarmi il numero
    num = max(num,v);
}
num = num+1;
choosing = false;
for j in 1 .. N except i { //backery
    do{
        send(Pj,choosing);
        receive(Pj,v);
    }while (v == true);
    do{
        send(Pj,v);
        receive(Pj,v);
    }while (v != 0 && (v,j) < (num,i));
}
/*CS*/
num = 0;
}

```

```

//global variable
//inizialization:
int num = 0;
boolean choosing = false;
// and process ip/ports

```

```

while (1){ //server thread
    receive(Pj,message);
    if (message is a number)
        send(Pj,num);
    else
        send(Pj,choosing);
}

```

Assumptions:

- Finite response time
- Reliable communication channels